Pertemuan 2

Manajemen Proses

Copyright © September 2026

Materi Pertemuan 2

- Konsep proses dan thread
- Penjadwalan proses dan algoritma penjadwalan
- Deadlock: penyebab, pencegahan, dan penanganan

Copyright © September 2026

Konsep Proses dan Thread

Copyright © September 2026

PROSES

Proses adalah program yang sedang dieksekusi, dan merupakan unit dasar dari eksekusi di dalam sistem operasi. Proses memiliki beberapa komponen penting, termasuk:

1. **Kode Program** : Instruksi yang akan dieksekusi.
2. **Data** : Variabel dan struktur data yang digunakan oleh program.
3. **Stack** : Menyimpan data lokal dan variabel fungsi.
4. **Heap** : Memori dinamis yang dialokasikan selama runtime.
5. **Process Control Block (PCB)**: Struktur data yang menyimpan informasi tentang proses, seperti status, program counter, register CPU, dan informasi manajemen memori.

Copyright © September 2026

PROSES (LANJUTAN…)

Proses dapat berada dalam beberapa keadaan, termasuk:

1. **New** : Proses sedang dibuat.
2. **Running** : Instruksi sedang dieksekusi.
3. **Waiting** : Proses menunggu sebuah kejadian atau sumber daya.
4. **Ready** : Proses siap untuk dieksekusi oleh CPU.
5. **Terminated**: Proses telah selesai eksekusi.

Copyright © September 2026

PROSES (LANJUTAN…)

Proses dapat berada dalam beberapa keadaan, termasuk:

1. **New** : Proses sedang dibuat.
2. **Running** : Instruksi sedang dieksekusi.
3. **Waiting** : Proses menunggu sebuah kejadian atau sumber daya.
4. **Ready** : Proses siap untuk dieksekusi oleh CPU.
5. **Terminated**: Proses telah selesai eksekusi.

Copyright © September 2026

THREAD

Thread (atau utas) adalah unit dasar dari eksekusi yang lebih kecil dari proses. Setiap proses memiliki setidaknya satu thread, yang disebut thread utama. Sebuah thread berbagi sebagian besar sumber daya dengan thread lain dalam proses yang sama, termasuk:

1. **Kode Program**
2. **Data**
3. **Heap**

Namun, setiap thread memiliki:

1. **Thread Control Block (TCB)**: Struktur data yang menyimpan informasi tentang thread, seperti program counter, register, dan stack pointer.
2. **Stack**: Menyimpan data lokal dan variabel fungsi untuk thread tersebut.

Copyright © September 2026

PERBEDAAN ANTARA PROSES DAN THREAD

1. Memori dan Sumber Daya

- **Proses**: Setiap proses memiliki ruang alamat memori yang terpisah. Proses tidak berbagi memori dengan proses lain.
- **Thread**: Thread dalam proses yang sama berbagi ruang alamat memori yang sama. Ini memungkinkan komunikasi antar-thread lebih cepat dan lebih efisien, tetapi juga dapat menyebabkan masalah sinkronisasi.

2. Pembuatan dan Penghancuran

- **Proses**: Membuat dan menghancurkan proses memerlukan lebih banyak overhead karena harus mengalokasikan dan mengelola ruang alamat memori yang terpisah.
- **Thread**: Membuat dan menghancurkan thread lebih ringan dan cepat karena thread berbagi sebagian besar sumber daya dengan thread lain dalam proses yang sama.

Copyright © September 2026

PERBEDAAN ANTARA PROSES DAN THREAD

3. Komunikasi Antar-Unit

Proses: Komunikasi antar-proses (IPC) memerlukan mekanisme khusus seperti message passing, shared memory, atau pipes.

Thread: Komunikasi antar-thread lebih mudah karena thread dalam proses yang sama dapat mengakses data bersama secara langsung. Namun, ini juga menimbulkan tantangan dalam sinkronisasi.

4. Keamanan dan Stabilitas

Proses: Karena setiap proses memiliki ruang alamat memori yang terpisah, kegagalan dalam satu proses biasanya tidak mempengaruhi proses lain.

Thread: Karena thread berbagi ruang alamat memori yang sama, kesalahan dalam satu thread dapat mempengaruhi thread lain dalam proses yang sama, yang dapat menyebabkan ketidakstabilan proses.

Copyright © September 2026

CONTOH PENERAPAN

Proses: Sebuah aplikasi seperti browser web adalah contoh dari sebuah proses. Setiap jendela atau tab browser dapat berjalan sebagai proses terpisah dalam beberapa implementasi untuk meningkatkan stabilitas.

Thread: Di dalam browser web, proses utama dapat memiliki beberapa thread untuk menangani rendering halaman, pemutaran video, atau pengunduhan file secara bersamaan.

Copyright © September 2026

MANFAAT MENGGUNAKAN THREAD

1. ***Parallelism:*** Thread memungkinkan program untuk melakukan banyak tugas sekaligus, yang dapat meningkatkan performa, terutama pada sistem multi-core.

2. ***Responsiveness:*** Aplikasi dapat tetap responsif saat melakukan operasi latar belakang, seperti mengunduh data atau memproses file besar.

3. ***Efisiensi Sumber Daya:*** Thread lebih ringan daripada proses karena mereka berbagi banyak sumber daya..

Copyright © September 2026

TANTANGAN PENGGUNAAN THREAD

1. ***Sinkronisasi:*** Mengelola akses ke data bersama antara thread memerlukan sinkronisasi yang hati-hati untuk mencegah kondisi balapan (race conditions).

2. ***Debugging:*** Debugging masalah thread bisa lebih kompleks daripada debugging proses tunggal.

3. ***Overhead Manajemen:*** Meskipun lebih ringan daripada proses, thread masih memerlukan manajemen yang dapat menambah overhead dalam aplikasi.

Copyright © September 2026

PENJADWALAN PROSES DAN ALGORITMA PENJADWALAN PROSES

Copyright © September 2026

PENJADWALAN PROSES

Penjadwalan proses adalah mekanisme yang digunakan oleh sistem operasi untuk menentukan urutan eksekusi proses di CPU. Penjadwalan sangat penting untuk efisiensi dan performa sistem, karena mempengaruhi waktu respons, throughput, waktu putar balik, dan utilisasi CPU

Copyright © September 2026

PENJADWALAN PROSES

Penjadwalan Proses

1. **Dispatcher**: Komponen sistem operasi yang mengalihkan kontrol CPU dari satu proses ke proses lainnya.
2. **Context Switch**: Proses menyimpan keadaan dari proses saat ini dan memuat keadaan dari proses berikutnya yang akan dieksekusi.

Copyright © September 2026

PENJADWALAN PROSES

Penjadwalan Proses

3. Criteria for Scheduling:

1. **CPU Utilization**: Memaksimalkan waktu CPU aktif.
2. **Throughput**: Jumlah proses yang selesai per satuan waktu.
3. **Turnaround Time**: Waktu dari pengajuan proses hingga selesai.
4. **Waiting Time**: Waktu total proses menunggu di antrian ready.
5. **Response Time**: Waktu dari pengajuan permintaan hingga respons pertama dihasilkan.

Copyright © September 2026

ALGORITMA PENJADWALAN

1. First-Come, First-Served (FCFS)

Deskripsi: Proses dijadwalkan sesuai urutan kedatangan.

Kelebihan: Mudah diimplementasikan.

Kekurangan: Waktu tunggu bisa sangat lama untuk proses yang datang belakangan (efek konvoi).

Contoh: Jika proses P1, P2, P3 datang berturut-turut, maka urutannya adalah P1, P2, P3.

Shortest Job Next (SJN) / Shortest Job First (SJF)

Copyright © September 2026

ALGORITMA PENJADWALAN

2. Shortest Job Next (SJN) / Shortest Job First (SJF)

Deskripsi: Proses dengan waktu eksekusi terpendek dijadwalkan terlebih dahulu.

Kelebihan: Meminimalkan waktu tunggu rata-rata.

Kekurangan: Sulit untuk mengetahui waktu eksekusi yang tepat sebelumnya. Bisa menyebabkan kelaparan (starvation).

Contoh: Jika proses P1 (8), P2 (4), P3 (2) datang, urutannya adalah P3, P2, P1.

Copyright © September 2026

ALGORITMA PENJADWALAN

3. Priority Scheduling

Deskripsi: Proses dijadwalkan berdasarkan prioritas. Proses dengan prioritas lebih tinggi dijalankan lebih dulu.

Kelebihan: Memungkinkan penanganan proses penting lebih cepat.

Kekurangan: Bisa menyebabkan kelaparan untuk proses berprioritas rendah. Solusi: Aging (meningkatkan prioritas proses yang sudah lama menunggu).

Contoh: Jika proses P1 (prioritas 2), P2 (prioritas 1), P3 (prioritas 3) datang, urutannya adalah P2, P1, P3.

Copyright © September 2026

ALGORITMA PENJADWALAN

4. Round Robin (RR)

Deskripsi: Setiap proses diberi waktu eksekusi yang sama (time quantum) secara bergiliran.

Kelebihan: Baik untuk sistem time-sharing. Memberikan waktu respons yang lebih baik.

Kekurangan: Efisiensi bergantung pada panjang time quantum. Terlalu besar: mirip FCFS, terlalu kecil: terlalu banyak context switching.

Contoh: Jika proses P1, P2, P3 datang dengan time quantum 4, maka urutannya adalah P1 (4), P2 (4), P3 (4), lalu kembali ke P1, P2, P3 sampai selesai.

Copyright © September 2026

ALGORITMA PENJADWALAN

5. Multilevel Queue Scheduling

Deskripsi : Proses dibagi ke dalam beberapa queue berdasarkan kategori (misalnya foreground dan background), masing-masing dengan algoritma penjadwalan sendiri.

Kelebihan : Mengizinkan penanganan yang berbeda untuk berbagai jenis proses.

Kekurangan : Sulit untuk mengatur perpindahan antara queue.

Contoh : Foreground menggunakan RR, background menggunakan FCFS.

Copyright © September 2026

ALGORITMA PENJADWALAN

6. Multilevel Feedback Queue Scheduling

Deskripsi : Mirip dengan multilevel queue, tetapi proses dapat berpindah antar queue berdasarkan perilaku dan waktu tunggu.

Kelebihan : Menyediakan fleksibilitas dan penyesuaian dinamis terhadap proses yang berbeda.

Kekurangan : Kompleksitas tinggi dalam implementasi dan pengaturan.

Contoh : Proses mulai di queue tingkat tinggi dengan quantum kecil, jika tidak selesai, pindah ke queue lebih rendah dengan quantum lebih besar

Copyright © September 2026

ALGORITMA PENJADWALAN

Contoh Ilustrasi

Misalkan ada tiga proses dengan waktu burst berikut:

P1: 24

P2: 3

P3: 3

FCFS:

Urutan: P1, P2, P3

Waktu tunggu: P1 = 0, P2 = 24, P3 = 27

Waktu tunggu rata-rata = (0 + 24 + 27) / 3 = 17

Copyright © September 2026

ALGORITMA PENJADWALAN

Contoh Ilustrasi

Misalkan ada tiga proses dengan waktu burst berikut:

P1: 24

P2: 3

P3: 3

SJF:

Urutan: P2, P3, P1

Waktu tunggu: P2 = 0, P3 = 3, P1 = 6

Waktu tunggu rata-rata = (0 + 3 + 6) / 3 = 3

Copyright © September 2026

ALGORITMA PENJADWALAN

Contoh Ilustrasi

Misalkan ada tiga proses dengan waktu burst berikut:

P1: 24

P2: 3

P3: 3

Round Robin (Quantum = 4):

Urutan: P1 (4), P2 (3), P3 (3), P1 (4), P1 (4), P1 (4), P1 (4)

Waktu tunggu: P1 = 9, P2 = 4, P3 = 7

Waktu tunggu rata-rata = (9 + 4 + 7) / 3 = 6.67

Copyright © September 2026

DEADLOCK: PENYEBAB, PENCEGAHAN, DAN PENANGANAN

 Copyright © September 2026

DEADLOCK

Deadlock adalah kondisi di mana dua atau lebih proses saling menunggu satu sama lain untuk melepaskan sumber daya yang dibutuhkan, sehingga tidak ada satu pun dari proses tersebut yang dapat melanjutkan eksekusi. Deadlock dapat menyebabkan sistem menjadi tidak responsif dan memerlukan intervensi manual untuk memulihkan keadaan normal.

Copyright © September 2026

PENYEBAB DEADLOCK

- Deadlock terjadi jika empat kondisi berikut terpenuhi secara simultan:

1.Mutual Exclusion (Eksklusi Mutual):

Hanya satu proses yang dapat menggunakan sumber daya dalam satu waktu.

Contoh: Sebuah printer yang hanya dapat digunakan oleh satu proses pada satu waktu.

Copyright © September 2026

PENYEBAB DEADLOCK

2. Hold and Wait (Menahan dan Menunggu):

Sebuah proses yang menahan satu atau lebih sumber daya sementara menunggu sumber daya lain yang sedang digunakan oleh proses lain.

Contoh: Proses A memiliki sumber daya X dan menunggu sumber daya Y yang sedang digunakan oleh proses B, sementara proses B memiliki sumber daya Y dan menunggu sumber daya X.

Copyright © September 2026

PENYEBAB DEADLOCK

3. No Preemption (Tidak Ada Preemption):

Sumber daya yang sedang digunakan tidak dapat dipaksa untuk dilepaskan; hanya dapat dilepaskan secara sukarela oleh proses yang memegangnya.

Contoh: Sebuah proses yang mencetak data tidak dapat diinterupsi sampai proses selesai.

Copyright © September 2026

PENYEBAB DEADLOCK

3. No Preemption (Tidak Ada Preemption):

Sumber daya yang sedang digunakan tidak dapat dipaksa untuk dilepaskan; hanya dapat dilepaskan secara sukarela oleh proses yang memegangnya.

Contoh: Sebuah proses yang mencetak data tidak dapat diinterupsi sampai proses selesai.

Copyright © September 2026

PENYEBAB DEADLOCK

4. Circular Wait (Menunggu Melingkar):

Terdapat satu set proses {P1, P2, ..., Pn} di mana P1 menunggu sumber daya yang dipegang oleh P2, P2 menunggu sumber daya yang dipegang oleh P3, dan seterusnya hingga Pn menunggu sumber daya yang dipegang oleh P1.

Contoh: Proses A menunggu sumber daya yang dipegang oleh proses B, proses B menunggu sumber daya yang dipegang oleh proses C, dan proses C menunggu sumber daya yang dipegang oleh proses A

Copyright © September 2026

PENCEGAHAN DEADLOCK

Pencegahan deadlock melibatkan penghilangan satu atau lebih dari empat kondisi yang menyebabkan deadlock.

1.Mutual Exclusion:

Tidak dapat dihilangkan secara praktis karena beberapa sumber daya memang harus digunakan secara eksklusif.

.

Copyright © September 2026

PENCEGAHAN DEADLOCK

2. Hold and Wait:

Proses harus meminta semua sumber daya yang dibutuhkan sekaligus sebelum memulai eksekusi.

Kekurangan: Mungkin menyebabkan efisiensi rendah karena proses mungkin menahan sumber daya lebih lama dari yang diperlukan.

Copyright © September 2026

PENCEGAHAN DEADLOCK

3. No Preemption:

Jika sebuah proses menahan beberapa sumber daya dan meminta sumber daya lain yang tidak tersedia, semua sumber daya yang dipegang oleh proses tersebut dapat dipreempted.

Kekurangan: Mungkin tidak selalu praktis untuk semua jenis sumber daya.

.

 Copyright © September 2026

PENCEGAHAN DEADLOCK

4. Circular Wait:

Menentukan urutan linier sumber daya dan meminta proses untuk meminta sumber daya dalam urutan yang meningkat.

Contoh: Jika sumber daya diberi nomor, proses harus meminta sumber daya dengan nomor yang lebih rendah sebelum meminta sumber daya dengan nomor yang lebih tinggi.

Copyright © September 2026

PENANGANAN DEADLOCK

Penanganan deadlock melibatkan deteksi dan pemulihan deadlock setelah terjadi atau menghindari kondisi yang menyebabkan deadlock.

.

Copyright © September 2026

PENANGANAN DEADLOCK

1. Deteksi Deadlock:

- Sistem secara periodik memeriksa apakah ada deadlock.
- Menggunakan algoritma deteksi deadlock untuk mencari siklus dalam graf alokasi sumber daya.

Contoh: Algoritma deteksi siklus pada graf.

Copyright © September 2026

PENANGANAN DEADLOCK

2. Pemulihan dari Deadlock:

- **Preemption**: Secara paksa mengambil sumber daya dari proses yang memegangnya.
- **Rollback**: Mengembalikan proses ke keadaan sebelumnya yang aman dan mencoba kembali.
- **Terminasi Proses**: Menghentikan satu atau lebih proses dalam siklus deadlock.
  - ✓ **Abort semua proses dalam deadlock**: Menghentikan semua proses yang terlibat dalam deadlock.
  - ✓ **Abort satu proses pada satu waktu**: Menghentikan satu proses dan memeriksa kembali apakah deadlock masih ada

Copyright © September 2026

PENANGANAN DEADLOCK

3. Penghindaran Deadlock:

- Menggunakan algoritma seperti Banker's Algorithm untuk memastikan bahwa sistem selalu dalam keadaan aman.
- Proses hanya diberikan sumber daya jika hal itu tidak menyebabkan keadaan tidak aman.

Contoh: Algoritma Banker yang mempertimbangkan jumlah sumber daya yang tersedia, sumber daya yang diminta oleh proses, dan sumber daya yang dipegang oleh proses.

Copyright © September 2026

REAL LIFE DEADLOCK DI KANTOR

Di sebuah kantor, dua karyawan, A dan B, ingin mencetak dan mengirim faks dokumen. Printer dan mesin faks adalah sumber daya bersama yang harus digunakan oleh kedua karyawan.

- Karyawan A menahan printer dan menunggu mesin faks.
- Karyawan B menahan mesin faks dan menunggu printer.
- **Mutual Exclusion**: Printer dan mesin faks hanya dapat digunakan oleh satu karyawan pada satu waktu.
- **Hold and Wait**: Karyawan A dan B menahan sumber daya yang mereka miliki dan menunggu sumber daya lainnya.
- **No Preemption**: Sumber daya yang digunakan tidak dapat diambil paksa dari karyawan yang menggunakannya.
- **Circular Wait**: Karyawan A menunggu mesin faks yang dipegang oleh karyawan B, dan karyawan B menunggu printer yang dipegang oleh karyawan A.

Copyright © September 2026

Catatan

Dosen dapat memberikan contoh berupa studi kasus lainnya terkait deadlock di Real Life dan membahasnya bersama dengan mahasiswa.

 Copyright © September 2026

KESIMPULAN

1. Memahami konsep proses dan thread adalah penting untuk mengoptimalkan performa dan efisiensi aplikasi serta untuk memastikan stabilitas dan keamanan dalam sistem operasi modern.
2. Algoritma penjadwalan memungkinkan sistem operasi dapat memilih yang paling sesuai dengan kebutuhan dan karakteristik beban kerja yang dihadapi, untuk mencapai kinerja dan efisiensi optimal.
3. Dengan memahami penyebab, pencegahan, dan penanganan deadlock, sistem operasi dapat mengelola sumber daya secara lebih efisien dan mengurangi risiko terjadinya deadlock yang dapat mempengaruhi kinerja dan stabilitas sistem.

Copyright © September 2026